# Proyecto Final — Mario Bros con aprendizaje por refuerzo


- Sahian Salomé Gutiérrez Ossa.
- David Alejandro Gutiérrez Leal.
- Mariamny Del Valle Ramírez Telles.


## 3.1 Selección del problema

El problema es el **playtesting automático**: un agente que juega un nivel por su cuenta para que un diseñador sepa dónde se atasca o muere un jugador, sin tener que reunir personas para probarlo. Super Mario Bros 1-1 es el caso de estudio.

**Contexto.** Probar un nivel con personas es lento y caro, y cada cambio de diseño obliga a repetir la prueba. Un agente entrenado puede jugar cientos de partidas en minutos y reportar siempre las mismas métricas.

**Usuarios o clientes.**

- Diseñadores de niveles de estudios pequeños, que no tienen un equipo de QA.
- Equipos de QA que quieren detectar zonas imposibles o bloqueos antes de que llegue una persona.
- Docentes y estudiantes, como demostración de cómo aprende un agente.

**Tarea que apoya el modelo.** Responder tres preguntas sobre un nivel: ¿se puede completar?, ¿hasta dónde llega el agente? y ¿dónde muere con más frecuencia? Esas respuestas salen directamente de las métricas `completed`, `progress` y `death_x`.

**Origen de los datos.** No hay un dataset fijo: los datos se generan al jugar. El emulador de NES (`nes-py` + `gym-super-mario-bros`) entrega en cada paso la imagen de la pantalla, una recompensa y un diccionario `info` con `x_pos`, `status`, `flag_get`, `life` y `time`.

**¿Por qué aprendizaje por refuerzo?** No existe una etiqueta con la "acción correcta" en cada pantalla. Solo se sabe, después de actuar, si la decisión acercó a Mario a la meta o lo mató. Ese es exactamente el escenario de RL: aprender por prueba y error a partir de recompensas, con decisiones secuenciales en las que cada acción cambia el estado siguiente.

## Marco teórico: de Q-Learning a DQN y PPO

El proyecto aplica el paradigma de refuerzo del curso: DQN es Q-Learning con una red neuronal en lugar de la tabla Q.

### Mario como un MDP

| Elemento del MDP | En el proyecto |
|---|---|
| Estados S | 4 frames apilados en gris de 84×84, observación `(4, 84, 84)` |
| Acciones A | 7 acciones de `SIMPLE_MOVEMENT` (NOOP, caminar a la derecha, saltar hacia la derecha, correr hacia la derecha, saltar corriendo a la derecha (salto más largo), saltar en el lugar, caminar a la izquierda) |
| Transición P(s′ \| s, a) | El emulador. Es desconocida para el agente, por eso se usa un método *model-free* |
| Recompensa R | Avance en x, penalización por tiempo, −15 al morir, bonos de `env-v2` |
| Descuento γ | 0.99 en DQN y PPO |

**Propiedad de Markov.** Un solo frame no la cumple: no muestra si Mario sube o baja, ni a qué velocidad va. Apilar 4 frames hace que el estado contenga la velocidad y la dirección, y así el estado actual vuelve a resumir lo necesario del pasado.

### Por qué no basta la tabla Q

Q-Learning clásico guarda una celda por cada par (estado, acción) y actualiza con la ecuación TD (Regla de Actualizacion de Q-learning). Con 4×84×84 píxeles de 256 valores, el número de estados posibles es astronómico y la gran mayoría nunca se repite. Una tabla no cabe en memoria ni generaliza entre pantallas parecidas.

### DQN: Q-Learning con una red

DQN reemplaza la tabla por una red Q(s, a; θ) que recibe la imagen y devuelve los 7 valores Q. Conserva todo lo demás de Q-Learning: es basado en valores, *model-free* y *off-policy*, usa el objetivo TD y explora con ε-greedy (de 1.0 a 0.05). La política se extrae con argmax: π(s) = argmax_a Q(s, a).

Agrega dos piezas para que el entrenamiento sea estable con una red:

- **Replay buffer** (100 000 transiciones): entrena con muestras al azar del pasado y no con pasos consecutivos, que están muy correlacionados.
- **Red objetivo** (actualizada cada 10 000 pasos): calcula el término γ·max Q(s′, a′) con una copia fija, para que el objetivo no se mueva en cada actualización.

Arquitectura (NatureCNN de SB3): 3 capas convolucionales (32×8×8 con stride 4, 64×4×4 con stride 2, 64×3×3 con stride 1), una densa de 3136 a 512 y una salida de 512 a 7.

**PPO: un método basado en políticas**

La red devuelve directamente π(a | s), una distribución sobre las 7 acciones, más un valor V(s) que ayuda a estimar qué tan buena fue cada acción. La acción se muestrea de π, así que la política es estocástica. Es *on-policy*: aprende solo de lo que acaba de jugar, con un recorte (*clip* 0.2) que impide cambiar demasiado la política en cada actualización.

## 3.2 Preprocesamiento de los datos

El preprocesamiento está implementado en `env.py` (`env-v2`) y es el mismo para todos los modelos, por eso sus resultados se pueden comparar. En RL no hay valores faltantes ni codificación de categorías: el equivalente es transformar la imagen y la recompensa.

| Paso | Qué hace | Por qué es necesario |
|---|---|---|
| Espacio de acciones | 256 combinaciones de botones → 7 acciones (`SIMPLE_MOVEMENT`) | Menos acciones = menos que explorar; las 7 bastan para completar el nivel |
| Frame skip 4 | Repite cada acción 4 frames y suma las recompensas | Frames seguidos casi no cambian; acelera 4 veces y da decisiones con efecto visible |
| Escala de grises | RGB (3 canales) → 1 canal | El color no cambia la decisión; reduce la entrada a un tercio |
| Redimensionado a 84×84 | 240×256 → 84×84 (`INTER_AREA`) | Unas 9 veces menos píxeles; conserva la forma de los objetos |
| Frame stacking 4 | Apila los últimos 4 frames → `(4, 84, 84)` | Da velocidad y dirección; recupera la propiedad de Markov |
| Escalado de píxeles | `uint8` [0, 255] → [0, 1] (lo hace SB3) | Entradas pequeñas estabilizan el gradiente; se guarda `uint8` para ahorrar memoria |
| No-ops aleatorios | 0 a 30 NOOP al reiniciar | Cada episodio empieza distinto; sin esto el juego es determinista y el agente memoriza |
| Recompensa con shaping | +500 bandera, +100 hongo o flor, −10 perder el poder | Para que Mario analice qué hacer para ganar más puntos y que no hacer para evitar perder los que tiene acumulados |
| Escalado de la recompensa | ×0.01, solo para el algoritmo (`wrappers.py`) | Con recompensas de hasta ±60 por paso, los valores Q llegan a miles y el aprendizaje se vuelve inestable |

**Justificación del shaping.**

- El hongo (+100) vale más de lo que cuesta perder el poder (−10).
- La bandera vale 5 veces el hongo(+500), porque es el objetivo.
- Perder el poder cuesta menos que morir (−15), y morir no cobra además la pérdida de poder.
- No se premian enemigos ni cajas.

**Escalado y métricas.** El escalado no cambia lo que se reporta: va por fuera de `Monitor`, así que `episodes.csv` y `evaluate.py` siguen usando la recompensa sin escalar. Esto es análogo a escalar solo las variables que entran al modelo en aprendizaje supervisado.

## 3.3 Análisis exploratorio adaptado a RL

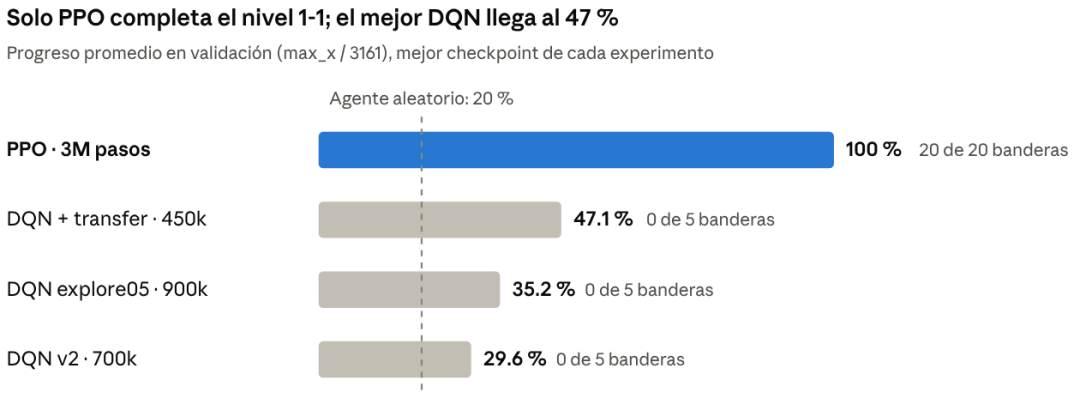
| Análisis | Fuente | Hallazgo | Decisión que motivó |
|---|---|---|---|
| Agente aleatorio en 1-1 | `results/val/random_1-1_*` | `max_x` promedio 632, 0 % de banderas | Es la línea base: un modelo que no la supere no aprendió nada útil |
| Escala de la recompensa | `env.py`, `info["reward_env"]` | Hasta ±15 por frame, ±60 por decisión, más bonos de +500 | Escalar la recompensa ×0.01 |
| Determinismo del juego | Smoke test con no-ops | Misma semilla = mismo episodio | Agregar 0–30 no-ops al reiniciar |
| Posiciones de muerte | `death_x` en `results/val/` | Se repiten ~310, ~700 y ~1125 | Son obstáculos concretos; sirven para explicar fallos y para el playtesting |
| Curva de entrenamiento DQN | `runs/dqn/dqn_v2_s1/episodes.csv` | `max_x` estancado entre 500 y 700 durante 800k pasos | Probar más exploración, transfer learning y PPO |
| Longitud de los niveles | `levels.json` | 1-1: 3161; 2-1 y 3-1: 3193 | Define `progress = max_x / longitud` |

## 3.4 Paradigma y modelado

| | DQN | PPO |
|---|---|---|
| Familia | Basado en valores (Q-Learning profundo) | Basado en políticas (actor-crítico) |
| Salida de la red | 7 valores Q(s, a) | π(a \| s) para 7 acciones + V(s) |
| Cómo decide | argmax Q con ε-greedy | Muestrea de π |
| Datos con los que aprende | *Off-policy*: replay buffer de 100 000 | *On-policy*: último rollout de ~2048 transiciones |
| Entornos en paralelo | 1 | 8 (`SubprocVecEnv`) |
| Código | `dqn/train.py` | `ppo/train.py` |
| Rol en el proyecto | Extensión directa de Q-Learning | Comparación y modelo desplegado |

## 3.5 Metodología de entrenamiento y selección del modelo

El modelo final es PPO con 3M pasos: es el único que completa el nivel, y lo hace en el 100 % de los episodios. Se compararon cuatro alternativas con el mismo entorno (`env-v2`) y el mismo protocolo de evaluación.




results/val/*.json y salidas de notebooks/transfer_learning_mario.ipynb · DQN: 5 episodios (semillas 100–104); PPO: 20 episodios (100–119)

### Alternativas y por qué se probaron

1. **DQN v2** (`exploration_fraction` 0.2, learning rate 1e-4, 800k pasos). Es la línea base del curso. Osciló mucho entre checkpoints y su curva de entrenamiento se estancó entre 500 y 700 de `max_x`.
2. **DQN explore05** (1M pasos). Prueba si más exploración saca al agente del estancamiento. Mejoró en 900k, pero colapsó en 1000k (`max_x` 39). Falta confirmar su configuración.
3. **DQN con transfer learning** (500k pasos). Copia las convoluciones de un PPO preentrenado de Hugging Face. Entrena primero con la CNN congelada (50k pasos, LR 1e-4) y después todo con LR 1e-5. Es el mejor DQN.
4. **PPO** (3M pasos, 8 entornos en paralelo, recompensa ×0.01). Es un método basado en políticas y el único que llega a la bandera.

### Cómo se eligió cada checkpoint

Cada experimento guarda copias del modelo cada 100k pasos (cada 250k en PPO). Se evalúan todas en validación y se elige la de mayor progreso promedio, no la última. La de DQN explore05 muestra por qué: la última copia fue la peor.

### Hiperparámetros principales

| Hiperparámetro | DQN | PPO | Justificación |
|---|---|---|---|
| Learning rate | 1e-4 fijo | 2.5e-4 → 2.5e-5 lineal | Valores estándar en Atari; el decaimiento evita cambios grandes al final |
| γ | 0.99 | 0.99 | Mario necesita planear varios segundos adelante |
| Exploración | ε de 1.0 a 0.05 en el 20 % de los pasos | Entropía 0.01 | Mucha exploración al inicio, explotación al final |
| Datos por actualización | Buffer de 100 000, lote de 32 | Rollout de ~2048, lote de 256, 4 épocas | Lo estándar de cada algoritmo |
| Red objetivo / clip | Cada 10 000 pasos | Clip 0.2 → 0.1 | Estabilizan el aprendizaje |
| Escala de la recompensa | 1.0 | 0.01 | Sin escalar, una decisión puede valer hasta ±60 y la bandera +500, y los valores que aprende la red se vuelven muy grandes e inestables. Con ×0.01 quedan cerca de ±1, como en Atari. Solo lo ve el algoritmo: las métricas siguen sin escalar. DQN v2 se entrenó sin escalar (1.0). |

**Por qué PPO es la opción final:** es el único que cumple la tarea del usuario (completar el nivel), con resultados idénticos en validación y prueba, y su entrenamiento fue estable, sin colapsos entre checkpoints.

## 3.6 Generalización

### Separación de datos

En RL no se divide un dataset, pero la lógica de entrenamiento, validación y prueba es la misma. Cada conjunto se usa para una sola cosa:

| Conjunto | Qué es | Para qué se usa |
|---|---|---|
| Entrenamiento | Nivel 1-1, semilla de entrenamiento 1 | Aprender los pesos |
| Validación | 1-1, semillas de evaluación 100–119 | Elegir el checkpoint y los hiperparámetros |
| Prueba en nivel visto | 1-1, semillas 200–209 | Reportar el resultado final, una sola vez |
| Prueba en niveles no vistos | 2-1 y 3-1 | Medir generalización real, una sola vez al final |

Cada semilla cambia cuántos no-ops se aplican al inicio (0 a 30), así que cada episodio arranca en un estado distinto.

### Evidencia de que no hay sobreajuste ni fuga

- **No hay sobreajuste a la validación:** PPO completa el 100 % en validación (20 de 20) y también en prueba (10 de 10), con semillas que no se usaron para elegir el modelo.
- **No hay fuga de información:** las semillas 200–209 y los niveles 2-1 y 3-1 no se usaron para elegir nada. Hay que mantenerlo así: nada de reentrenar ni ajustar en 2-1 o 3-1 antes de reportar.
- **Limitación honesta:** las semillas de 1-1 solo cambian el arranque, no el nivel. Miden robustez dentro del mismo nivel, no generalización a niveles nuevos.

### Qué esperar en 2-1 y 3-1

Es probable que el rendimiento baje mucho: 2-1 tiene otra paleta y otros enemigos, y el agente solo vio 1-1, así que probablemente podamos ver a Mario avanzando pero, si se encuentra con algo que no vio en el primer nivel, no podrá actuar ni superar el obstáculo de la manera correcta.

## 3.7 Métricas de desempeño

La métrica principal es el progreso; la recompensa total no se usa para comparar modelos.

| Métrica | Cómo se calcula | Pregunta del usuario que responde |
|---|---|---|
| `progress` (principal) | `max_x / longitud del nivel` | ¿Qué parte del nivel puede superar un jugador? |
| `completed` (tasa de completado) | 1 si `flag_get` fue verdadero | ¿Se puede terminar el nivel? |
| `death_x` | Posición x al morir | ¿Dónde está el obstáculo más difícil? |
| `max_x` | Máxima posición alcanzada | Igual que `progress`, en píxeles |
| `steps` | Decisiones del episodio | ¿Cuánto tarda en avanzar o en morir? |
| Media ± desviación estándar | Sobre todas las semillas | ¿El resultado es estable o depende de la suerte? |

**Por qué el progreso y no la recompensa.**

- La recompensa incluye bonos de shaping, que son pistas para aprender y no el objetivo.
- Cambia entre `env-v1` y `env-v2`, así que no se puede comparar entre versiones.
- Un agente puede subir su recompensa haciendo trampa (*reward hacking*) sin jugar mejor.

## 3.8 Despliegue e interfaz

Para la divulgación, auditoría y evaluación visual del proyecto se desarrolló una aplicación interactiva en **Streamlit** (app/main.py), concebida tanto para la inspección técnica de métricas como para la demostración ante usuarios no técnicos.

La aplicación fue desplegada públicamente en la nube a través de **Streamlit Cloud** y está disponible en:

https://mario-bros-reinforcement-learning-ptlu26zjorgag6xbfxsc6z.streamlit.app

La aplicación integra las siguientes funcionalidades:

1. Selección de agentes: Presenta los cuatro modelos implementados, destacando a PPO (3M pasos) como el mejor, con un 100 % de éxito (20/20 partidas), seguido de DQN Advanced, DQN CNN Base y el agente aleatorio como referencia.

2. Panel de métricas: Muestra indicadores visuales de rendimiento, como progreso promedio, récord alcanzado, recompensas acumuladas, llegada a la bandera y puntos habituales de muerte.

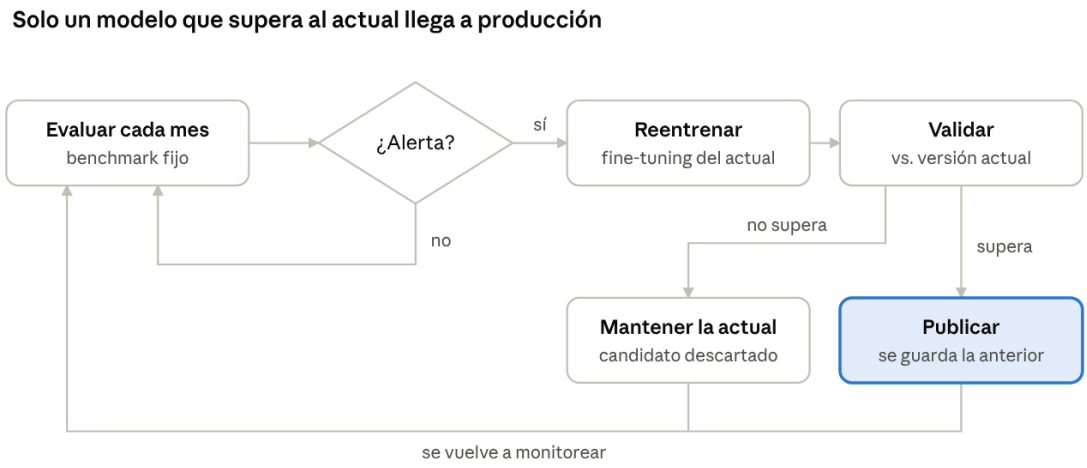
3. Visualización de partidas: Incorpora videos demostrativos en formato MP4 (H.264), con almacenamiento local y respaldo mediante Google Drive para garantizar su disponibilidad.

4. Ejecución y comparación: Incluye un modo en vivo para ejecutar agentes en entornos compatibles y gráficos comparativos que permiten analizar las diferencias de rendimiento entre los modelos.

En conjunto, la aplicación facilita la demostración, auditoría y evaluación visual del proyecto, presentando los resultados técnicos mediante una interfaz accesible tanto para usuarios especializados como para público general.

## 3.9 Estrategia de reajuste

El modelo se reajusta solo cuando una evaluación periódica dispara una alerta, y la versión nueva se publica solo si supera a la actual con el mismo protocolo.




ciclo de reajuste · 5 pasos, 2 decisiones

### Qué puede cambiar en producción

En este problema, los "datos nuevos" son niveles nuevos o modificados por los diseñadores.

### Cómo se detecta

- **Benchmark fijo:** cada mes, el modelo en producción juega 1-1 y los niveles de referencia con las mismas semillas (200–209). Como el protocolo no cambia, una caída solo puede venir del modelo o del entorno.
- **Alerta por desempeño:** el progreso promedio cae más de 10 puntos porcentuales frente a la última medición aceptada, o el completado en 1-1 baja del 90 %.
- **Alerta por cambio de datos:** llega un nivel nuevo donde el progreso queda por debajo del 30 %, o se actualiza una librería del entorno.
- **Señal de los usuarios:** diseñadores que reportan que el agente muere en zonas que una persona supera sin problema.

### Cómo se reentrena

Se hace fine-tuning desde el modelo actual con un learning rate bajo, agregando los niveles nuevos al entrenamiento.

### Cómo se valida y se publica

1. El candidato se evalúa con el mismo protocolo que el modelo actual: semillas 100–119 para validar y 200–209 para la prueba final.
2. Se publica solo si mejora en el nivel nuevo y no empeora en 1-1. Esta comparación se conoce como *champion vs. challenger*.
3. El modelo anterior se conserva para volver a él (*rollback*) si aparece un problema.
4. Cada versión queda trazable: `config.json` y los CSV ya registran `env_version` y `git_commit`.